

Practical Skills

BMED365: Topic List F01–F14

BMED365

Spring 2026

- 1 Jupyter Notebooks and Google Colab ([Jupyter](#), [Colab](#))
- 2 Python Fundamentals ([NumPy](#), [Pandas](#), [Matplotlib](#))
- 3 Machine Learning with scikit-learn ([scikit-learn](#))
- 4 Network Analysis with NetworkX ([NetworkX](#))
- 5 Deep Learning with PyTorch ([PyTorch](#))
- 6 AI Tools and LaTeX ([ChatGPT](#), [Gemini](#), [Claude](#), [Overleaf](#))

F01: Run Jupyter Notebooks in Google Colab

What is Jupyter Notebooks?

- Interaktive dokumenter som kombinerer kode, tekst og visualiseringer
- Kjør Python-kode celle for celle
- Standard i datavitenskapelig arbeid

Google Colab:

- Gratis sky-basert Jupyter-miljø fra Google
- Ingen installasjon – kjører i nettleseren
- Gratis GPU-tilgang (important for deep learning!)
- Integrert med Google Drive

Kom i gang:

- 1 Gå til colab.research.google.com
- 2 Logg inn med Google-konto
- 3 "New notebook" or åpne fra GitHub

Tip

Aktiver GPU: Runtime → Change runtime type → GPU

F02: Use Python variables, lists, and simple functions

Variabler:

```
alder = 45           # int
navn = "Pasient_A"   # str
risiko = 0.73        # float
er_syk = True        # bool
```

Lister:

```
symptoms = ["hodepine", "kvalme", "tretthet"]
valueer = [1.2, 3.4, 5.6, 7.8]
symptoms.append("feber") # Legg til element
```

Funksjoner:

```
def beregn_bmi(vekt, hoyde):
    return vekt / (hoyde ** 2)

bmi = beregn_bmi(70, 1.75) # -> 22.9
```

F03: Import and use libraries (numpy, pandas, matplotlib)

Import av biblioteker:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

NumPy:

- Numeriske beregninger
- Arrays og matriser
- Matematiske funksjoner

```
arr = np.array([1, 2, 3])
mean = np.mean(arr)
```

Pandas:

- Datamanipulering
- DataFrames (tables)
- CSV, Excel I/O

```
df = pd.read_csv("data.csv")
df.head()
```

F04: Read and inspect datasets with pandas

Reading data:

```
df = pd.read_csv("patients.csv")
```

Inspecting data:

```
df.head()           # First 5 rows
df.info()           # Column types, null values
df.describe()       # Statistics for numeric columns
df.shape            # (number of rows, number of columns)
df.columns          # Column names
```

Filtering and selection:

```
# Select columns
df[["age", "diagnosis"]]

# Filter rows
elderly = df[df["age"] > 65]
```

F05: Train a simple model with scikit-learn

Typisk ML-workflow:

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1. Forbered data
X = df[["feature1", "feature2"]]
y = df["label"]

# 2. Del i training og test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 3. Lag og tren modell
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# 4. Evaluer
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
```

F06: Visualize results with matplotlib

Fundamental plotting:

```
import matplotlib.pyplot as plt

# Linjediagram
plt.plot([1, 2, 3, 4], [10, 20, 25, 30])
plt.xlabel("Tid"); plt.ylabel("Verdi")
plt.title("Min_figur")
plt.show()
```

Histogram:

```
plt.hist(df["alder"], bins=20)
plt.xlabel("Alder")
plt.show()
```

Scatter plot:

```
plt.scatter(df["x"], df["y"])
plt.xlabel("X"); plt.ylabel("Y")
plt.show()
```

Seaborn

import seaborn as sns – Penere visualiseringer med simple kode!

F07: Bruke NetworkX for simple nettverksanalyse

Opprett og manipuler grafer:

```
import networkx as nx

# Opprett graf
G = nx.Graph()
G.add_nodes_from(["P1", "P2", "P3", "P4"])
G.add_edges_from([("P1", "P2"), ("P2", "P3"), ("P1", "P3")])

# Fundamental analyse
print(f"Antall_noder: {G.number_of_nodes()}")
print(f"Antall_kanter: {G.number_of_edges()}")

# Sentralitet
deg_cent = nx.degree_centrality(G)
print(f"Degree_centrality: {deg_cent}")

# Visualisering
nx.draw(G, with_labels=True, node_color="lightblue")
plt.show()
```

F08: Bygge og trene en modell med PyTorch

Enkel MLP i PyTorch:

```
import countsch
import countsch.nn as nn

class SimpleMLP(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super().__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        return out

model = SimpleMLP(784, 128, 10)  # MNIST-eksempel
```

Lab 2

Full training med loss, optimizer, og trainingsløkke gjennomgås i Lab 2.

F09: Use AI tools as coding assistants

Useful applications:

-  **Explain kode:** “Hva gjør denne funksjonen?”
-  **Feilsøking:** “Why får jeg feilmeldingen?”
-  **Fotherwiselayers:** “Gjør dette mer effektivt?”
-  **Generere kode:** “Skriv en funksjon...”

Tip for effective use:

- 1 Vær **spesifikk** i spørsgoalene
- 2 Inkluder **kontekst**
- 3 **Verifiser** alltid AI-kode
- 4 Bruk som **læringsverktøy**

AI-drevne IDE-er (Integrated Development Environment)

IDE = Edicounts + kompilacounts + debugger + verktøy i ett. **AI-drevet IDE** = IDE med LLM-agenter that can planlegge, skrive, teste og validere kode autonomt.

Betydning for medical/biomedical fotherwisekning: Senker terskelen to utvikle analyseverktøy, akselererer prototyping av ML-models, og muliggjør raskere oversettelse fra fotherwisekning til klinikk.

Cursor, Windsurf	Edicounts med agenter, plan mode
Google Antigraity	Agent-first, multi-modell (Gemini 3)
Replit Agent	Sky-basert IDE med AI-agent
Lovable	Naturlig språk → full-stack app
GitHub Copilot, JetBrains AI	AI-assistenter i eksisterende IDE-er

F10: Skrive dokumenter med LaTeX/Overleaf

BibTeX (.bib-fil):

```
@article{smith2024,  
  author = {Smith, J.},  
  title = {AI in Medicine},  
  journal = {Nature Med.},  
  year = {2024},  
  volume = {30},  
  pages = {123--130},  
  doi = {10.1038/s41591-  
    024-01234-5}  
}  
  
@book{bishop2006,  
  author = {Bishop, C.},  
  title = {Pattern Recog.},  
  publisher = {Springer},  
  year = {2006},  
  isbn = {978-0387310732}  
}
```

I tekst: `\cite{smith2024}`

IMRAD-struktur:

```
\documentclass{article}  
\usepackage{graphicx,booktabs}  
\begin{document}  
\title{Tittel}  
\author{Forfatter}  
\maketitle  
\begin{abstract} ... \end{abstract}  
% Innledning  
\section{Introduction}  
Bakgrunn og goal \cite{smith2024}.  
% Materiale og metoder  
\section{Methods}  
Datainnsamling og analyse.  
% Resultater  
\section{Results}  
\begin{table}[h]  
\begin{tabular}{lcc} \toprule  
Modell & AUC & F1 \\ \midrule  
CNN & 0.92 & 0.88 \\ \bottomrule  
\end{tabular}  
\caption{Resultater}\label{tab:1}  
\end{table}  
% Diskusjon  
\section{Discussion}  
Implikasjoner og limitasjoner.  
\bibliography{refs}  
\end{document}
```

Tidsskrift m/LaTeX:

- PLOS Medicine
- Nature / Nat. Med.
- Bioinformatics
- The Lancet
- BMJ
- Elsevier (mange)
- Springer Nature
- MDPI (open access)
- IEEE JBHI
- JMLR
- Frontiers

Overleaf Gallery

F11: Compose effective prompts for medical tasks

From Prompt Engineering to Context Engineering:

- **Prompt engineering:** Utforme gode spørsmål og instruksjoner
- **Context engineering:** Designe *hele konteksten* modellen bruker

Komponenter i context engineering:

- 1 **System prompt:** Rolle og fundamental instruksjoner
- 2 **Bruker-prompt:** Det spesifikke spørsmålet
- 3 **Hentet kontekst (RAG):** Journaler, retningslinjer, artikler
- 4 **Verktøy:** Laboratorysøk, drugdatabase, ICD-10
- 5 **Samtalehistorikk:** Tidligere dialog i konsultasjonen
- 6 **Strukturerte data:** JSON med lab-verdier, vitale tegn

Medical significance

Rik kontekst = mer relevante, personaliserte og trygge AI-svar

F12: Apply context engineering in practice

Example: Enkel prompt vs. rik kontekst

Enkel prompt

What is treatmenten for hjertesvikt?

→ Generelt svar, ikke personalisert

Rik kontekst

System: Clinical beslutningsstøtte

Pasient: 72 år, eGFR 45, Warfarin

RAG: ESC Guidelines 2023

Spørsmål: Behandlingsendringer?

→ Personalisert, evidensbasert svar

Ferdighetsnivåer:

Nybegynner:

Viderekommende:

Avansert:

Ekspert:

Prompt-formulering (rolle, format, eksempler)

Strukturerte system prompts med seksjoner

RAG-integrasjon og verktøybruk

Multi-agent orkestrering

F13: Choose the right LLM model for the task

Modellstrength for medicale taskr:

Oppgavetype	Anbefalt modell	Begrunnelse
Strukturert ekstraksjon	GPT-5.2	God instruksjonsetterlevelse
Etiske dilemmaer	Claude Opus 4.5	Nyansert resonnering
Image Analysis	Gemini 3	Multimodal fra grunn av
Lang journalanalyse	Claude Opus 4.5	200K tokens kontekst
Oppdatert fotherwisekning	Gemini 3	Integrert sanntidssøk
Verktøyintegrasjon	GPT-5.2	God API for funksjoner
Pasientkommunikasjon	Claude Opus 4.5	Empatisk, patientsentrert
Beregninger	Gemini 3	Sterk på vitenskap

Practical recommendation

For critique taskr: Test på flere models og sammenlign output

F14: Compare output from different LLMs

Evaluationskriterier for medical LLM-output:

Kvalitetsdimensjoner:

- ❶ **Medical accuracy**
Er informasjonen korrekt?
- ❷ **Usikkerhetshåndtering**
Uttrykkes usikkerhet passende?
- ❸ **Strukturert output**
Følges format-instruksjoner?
- ❹ **Sikkerhetsatferd**
Henvises til lege ved behov?
- ❺ **Clinical utility**
Er svaret praktisk anvendbart?

Praktisk progresssmåte:

- ❶ Definer testcase med fasit
- ❷ Send samme prompt til 2–3 models
- ❸ Vurder mot kriteriene (1–5 scale)
- ❹ Dokumenter strengthr/weaknesser
- ❺ Velg modell for produksjon

Lab 3 Notebook 04

Inneholder ferdige eksperimenter for modellsammenligning

Summary: Practical Skills

Jupyter og Python:

- F01–F04: Colab, variabler, lister, funksjoner, pandas

Machine Learning:

- F05–F06: scikit-learn workflow, matplotlib visualisering

Spesialisert:

- F07: NetworkX for nettverksanalyse
- F08: PyTorch for deep learning

Verktøy og LLM:

- F09–F10: AI-assistenter, LaTeX/Overleaf
- F11–F14: Context engineering, modellvalg, prompt-optimalisering

Lynkurs og Labs

Alle disse ferdighetene praktiseres gjennom Lynkurset og Lab 0–3. Øving gir mestring!